

LAB 11: State Machine

Purpose

In the last lab, you got your mechanism to move the flashlight to different positions. In this lab, we will first add the missing functionality to interact with the playing field and to go through all the steps of hitting the photosensors to move the targets back. We will then give you several possible extensions to your code that you can use to improve the performance of your mechanism and score higher in the final competition.

Communication with the playing field

Your mechanism will communicate with the playing field through four analog signals – one for each sensor pointed at a target on a rail. They are connected to pins A0 (PIN_PROXSENSE1), A1 (PIN_PROXSENSE2), A2 (PIN_PROXSENSE3), and A3 (PIN_PROXSENSE4) on the Arduino.

Your controller will take information from the four proximity sensors through these analog signals to determine which flags are up. Then your controller must decide how to act on that information in order to hit the photosensors with light to send the targets backward.

The proximity sensors are highly sensitive to ambient light, and each sensor's calibration may differ slightly from the others. It is best practice to re-measure the proximity sensor readings and update these values in your Arduino code at the beginning of each session when you work with the playing field. Note that these values are set to 0 in the provided code template – these values will need to be updated before the code will run properly.

State Machine

Your mechanism must be able to do several things:

1. After you turn it on, it must calibrate itself to determine the zero for the digital encoder.
2. Then, it must determine which target is closest to reaching the end of the rail.
3. The mechanism must point the light at a target.
4. Loop back to step 2 and repeat.

In our code, we generate this behavior by means of a so-called "state machine". Your mechanism will be in one of three states: 1) *Calibrate*, 2) *Choose Active Target*, 3) *Move to Target*. For each of these states, we need to define a few things: what to do in the state, when to leave the state, and what to do next.

What to do in each state

If we are in the *Calibrate* state, we apply a constant voltage to make sure that the mechanism moves against the end stop with the limit switch. In each of the other states, we want the mechanism to move to a certain position.

During the *Choose Active Target* state, the controller uses the current proximity sensor readings along with the previous proximity sensor reading to determine the location and direction of each target. Then, the controller decides which photosensor to point at based on the location (minimize distance to rail end) and direction (must be moving forward).

During the *Move to Target* state, the controller must move the mechanism to the corresponding photosensor to send the zombie backwards.

How to transition between states

The mechanism will always start in the *Calibrate* state. Once the linkage has moved against the end stop with the limit switch, the limit switch will activate and the velocity will be brought to zero, due to the end stop. When this happens, we can calibrate and transition into the *Choose Active Target* state. The controller will determine which forward moving flag is closest to the end of the rail. It will then transition into the *Move to Target* state which is active until the desired position has been reached. Then, the controller will transition back into the *Choose Active Target* state.

Figure 1 shows a formal description of this process. Each state is represented by a box. The information in the box states what to do while the system is in that state. Arrows indicate transitions to other states. The label on the arrow tells you what will trigger the transition, and the text next to the arrow what to do during the transition.

State Machine Diagram

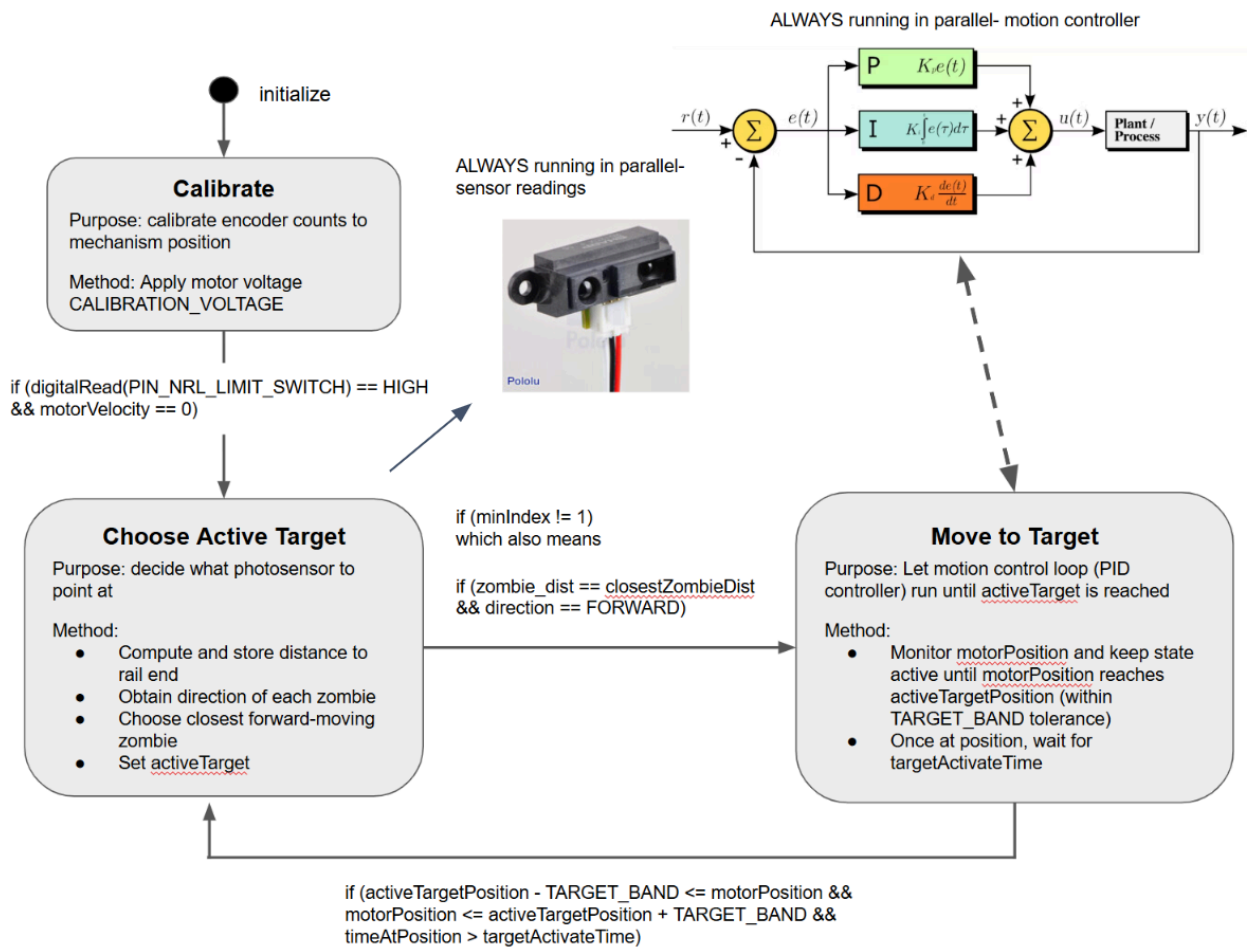


Figure 1. The state machine used in this project.

In our Arduino code, we implement this state machine as a switch / case statement (see this link for a code-reference:

<https://www.arduino.cc/reference/cs/language/structure/control-structure/switchcase/>).

```
switch (var) {
  case 1:
    //do something when var equals 1
    break;
  case 2:
    //do something when var equals 2
    break;
  default:
    // if nothing else matches, do the default
    // default is optional
    break;
}
```

In our application, the switching variable is simply the state itself. The variable is thus simply called 'state'. The code below shows the example of the *Move_To_Target* state case statement

as an example. All of these statements look very similar. In particular, in each of the case statements we do two things:

- 1) Set a *targetPosition* (exception: calibrate) [highlighted in green]
- 2) Run an *if* statement, that determines whether we should leave the state [highlighted in yellow]. If that statement is true, we might perform additional actions [none here] and then change the *state* variable to transit into the next state [highlighted in blue].

```
// *****/
/  
// In the MOVE_TO_TARGET state, we select an active target and move  
// toward it, or move toward Target 3 (a default position) if there  
// is no active target  
case MOVE_TO_TARGET:  
    desiredPosition = activeTargetPosition;  
    if (motorPosition <= activeTargetPosition + TARGET_BAND  
        && motorPosition >= activeTargetPosition - TARGET_BAND) {  
  
        state = CHOOSE_ACTIVE_TARGET;  
    }  
    break;
```

Understand & Finalize the State Machine

In order to get you accustomed to the state machine, we left a couple of blanks in the code ("[ME350_LinkageCode_V5.1.ino](#)") that you have to find and fill in:

- 1) Implement the *Calibrate* action. Since during the calibration, we will overwrite the voltage determined by the controller, this action is set outside the switch statement.

Find the lines of code that say:

```
if (state==CALIBRATE) {  
    // add calibration code here  
}
```

and add the line:

```
desiredVoltage = CALIBRATION_VOLTAGE;
```

- 2) Find a suitable negative value for CALIBRATION_VOLTAGE (which is set to 0 as default), this will move your mechanism to the leftmost point by applying a constant voltage (If your limit switch is on the right, the voltage must be positive).
- 3) If necessary, find a way to ensure you light up the target for long enough to send the zombie backwards (time delays, reading sensors, etc.)

With these changes, your code should be ready to run and you can start sending zombies backward. In the following we give you some guidelines for how to test without being connected to the playing field. If you have **not** wired for the playing field yet please refer to the section in Lab 3 "*Connect the wires for playing field signals*". Note: you do not need to insert the code snippet that we provided you at the end of lab 3.

After uploading your sketch, open the serial monitor to see what your Arduino is trying to do. The 2-letter codes are explained in comments near the end of the sketch.

Pressing the small button on the corner of the Arduino will restart it, executing the code from the beginning. Using this button is much faster than re-uploading code to restart the sketch.

You can make any changes to the code that you'd like. **Remember to back-up a copy of your code before making big changes!**

Improving your code

Congratulations, you got your mechanism to work! This concludes the formal part of the mechatronics labs.

Now it is time to improve and change your code to get better performance and higher scores. Rather than the “fill in the blanks” approach of the previous coding exercises, the following is more free-form. On the following pages, we give you a number of ideas that you could try to implement to make your system perform better. Some of them are easy to implement, some are more difficult. Some ideas might help all the teams and some might only help teams with specific issues. You can try as many as you want. Be creative and explore. Talk to your GSIs when you have any questions.

Re-calibrate

The problem: Because you either lose counts in the encoder or your motor connections are slipping, your mechanism starts to drift, and your mechanism is not reaching the targets after a couple of cycles.

Difficulty: EASY

The solution: If you have this problem, you can change the code in your state machine, such that instead of going from *Move to Target* to *Determine Active Targets*, it goes from *Move to Target* to *Calibrate*. That is, the calibration will be repeated after each target is lit up.

Adaptive Gains

The problem: When tuning your controller, you find that you perform better with different gains (KP, KD, and KI) at the different target positions.

Difficulty: EASY

The solution: You can make your gains state-dependent. First, remove the “const” in each line that defines a gain. Then find lines that start with

```
targetPosition = ...(and are not desiredPosition = activeTargetPosition;)
```

also look for lines starting with:

```
activeChutePosition = ...
```

Redefine your gain values right after each of these lines. This means simply stating

KP = 0.01; (or whatever value)

KD = 0.01; (or whatever value)

etc.

You can do this for one gain value, two values, or all of them. However, note that if you change a value in one state, you must change it in all states. Otherwise your code will use the last value that was defined, which could yield unpredictable results.

Smart Re-calibrate

The problem: Your mechanism calibration is drifting, but only slowly. You tried to re-calibrate after each cycle, but it really slows everything down.

Difficulty: MODERATE

The solution: Add a counter variable that increments by $+1$, every time a target is dropped. Leave the transition from *Move to Target* to *Determine Active Targets* as is, unless the counter has reached a certain value (maybe 5 cycles). In that case, transit from *Move to Target* to *Calibrate* and reset the counter to 0.

Expand the State Machine

The problem: Currently, the code only has one *Move to Target* state. That is, it is not able to distinguish between moving to different targets.

Difficulty: HARD

The solution: Add new states for each different target. Make sure to define this state at the top of the code. Model the code in the switch statement after the *Move to Target* state. The transition out of *Determine Active Target* should now depend on which target is active. This new state can use a different strategy (different *desiredPosition*, different controller gains) to get to the target.

Do ... Whatever...

The problem: While you acknowledge that your instructors are in principle well-meaning, you feel that none of their suggestions fits your personal needs (and those of your mechanism).

Difficulty: ???

The solution: Explore on your own. Add new states, add additional if/else statements, play with the controller, add a time dependency where your target position is a function of time, change the output of your code to reflect your personal style,... in short. Do whatever you like.

Appendix

Summary of Variables and CONSTANTS Available for Use – Pending Updates

Variable / CONSTANT	Units	Description
MOTOR – ENCODER RELATED		
motorPosition	encoder counts	Current motor position
targetPosition	encoder counts	Desired motor position
TARGET_BAND	encoder counts	"Close enough" range (+/- around target) when moving towards a target
encoderStatus	Binary	Past and Current A&B values of the encoder
desiredVoltage	Volt	Desired motor voltage
motorCommand	0-255	PWM signal sent to the motor
MOTOR VELOCITY RELATED		
motorVelocity	encoder counts per second	Current motor velocity
MIN_VEL_COMP_TIME	Microseconds	Minimal time that must pass between two velocity measurements
previousMotorPosition	encoder counts	Motor position the last time a velocity was computed
MIN_VEL_COMP_COUNT	encoder counts	Minimal change in motor position that must happen between two velocity measurements
TARGET RELATED		
activeTargetPosition	encoder counts	position of the currently active target
activeTargetIndex	array index	array index of the currently active target
WAIT_POS	Binary	True if linkage is moving to the wait position
SENSOR RELATED		
currVal	Prox sensor reading	Most recent sensor reading
prevVal	Prox sensor reading	Last updated value of sensor reading

prevChangeTime	milliseconds	Time at which the last update of the previous value occurred
pin	int	Arduino analog pin of the prox sensor
SENSOR NOISE RELATED		
alpha	unitless	Value weighting the previous sensor reading with the current sensor reading
noiseLimit	Prox sensor counts	Range in which prox sensor readings fluctuate when targets are stopped
upperNoiseLimit	Prox sensor counts	Noise limit for prox sensor readings on the upper end of prox sensor range
lowerNoiseLimit	Prox sensor counts	Noise limit for prox sensor readings on the lower end of prox sensor range
noiseThreshold	Prox sensor counts	Proximity sensor reading at which noiseLimit switches from upper to lower
CALIBRATION RELATED		
CALIBRATION_VOLTAGE	Volt	Motor voltage used during the calibration process
PID Controller		
KP	Volt / encoder counts	P-Gain
KI	Volt / (encoder counts * seconds)	I-Gain
KD	Volt * seconds / encoder counts	D-Gain
SUPPLY_VOLTAGE	Volt	Supply voltage at the H-Bridge
FRICTION_COMP_VOLTAGE	Volt	Positive voltage needed to overcome friction

Summary of Pin Assignments (Breadboard, Arduino, Pins)

Component	Pin	Breadboard	Arduino	Pull-down?
Limit Switch	NO Terminal	positive bus	\	\
	C terminal	row 1 pin a-e	pin 8	Yes
Toggle switch	on	positive bus	\	\
	off	row 8 pin a-e	pin 5	Yes
Encoder	GND	negative bus	\	\
	Vcc	positive bus	\	\
	A	\	pin 2	\
	B	\	pin 3	\
H-bridge	E12	\	pin 9	\
	I1	\	pin 10	\
	I2	\	pin 6	\
Playing field	Proximity Sensor 1 YELLOW	\	pin A0	\
	Proximity Sensor 2 YELLOW	\	pin A1	\
	Proximity Sensor 3 YELLOW	\	pin A2	\
	Proximity Sensor 4 YELLOW	\	pin A3	\
	Proximity Sensor 1-4 BLACK	negative bus	\	\
	Proximity Sensor 1-4 RED	positive bus	\	\

State Machine Diagram

